PROJECT REPORT · BACHELOR OF TECHNOLOGY (CSE) · 2026

# QuantSim

## A High-Frequency Trading Simulation Platform

Submitted in partial fulfilment of the requirements for the award of the degree of
**Bachelor of Technology in Computer Science & Engineering**


**Abhishek Anand**
Roll No. 22050440004

**Shakir Ahmad**
Roll No. 22050440090

**Amik Affan**
Roll No. 22050440017

Under the guidance of **Prof. Deepak Kumar Verma** — Project Guide & Head of Department, Dept. of CSE

**Department of Computer Science & Engineering**
Cambridge Institute of Technology, Tatisilwai, Ranchi – 835103, Jharkhand, India · 2026

OUTLINE

# Contents

MOTIVATION & PROBLEM STATEMENT

# No Open, End-to-End Quant Platform

### What exists today

Candle backtesters (Zipline, Backtrader)
with toy fills, no real machinery
OR closed firm stacks (Jane St, Citadel)
you never get to touch them
Nothing in between

### What's missing

One place to do the WHOLE job:
research · write code · backtest ·
build a portfolio · go live
Learn quant by doing, not reading
Real execution, real costs, real risk

### QuantSim

A free, open quant environment
as close to a real firm's stack
as we could build
Write your OWN strategies in the app
+ a library of built-ins to learn from

**The one-liner:** **the first free, open platform where you experience the entire quant pipeline — from research to development to portfolio to live — and write your own code to learn it for real.**

AIMS OF THE PROJECT

# Objectives

01 **Implement a high-performance C++ Limit Order Book and matching** engine with strict FIFO price-time priority and 8 order types.

02 **Build an event-driven backtester modelling probabilistic fills, queue** position, slippage, fees, rebates, borrow cost and configurable latency.

03 **Implement five canonical quantitative strategies with rigorous** mathematical foundations, plus a momentum baseline.

04 **Bridge the C++ core to Python via pybind11 with a research toolkit:** signal library, walk-forward and purged k-fold cross-validation.

05 **Develop a real-time GPU GUI (order book, equity/drawdown, liquidity** heatmap, Greeks surface) with an in-application C++ strategy compiler.

06 **Integrate a pre-trade risk subsystem and validate against live** Binance market data through a C++-native paper-trading harness.

EVERYTHING IN THE BOX

# A Full Quant Stack, Built In

### Matching Engine

FIFO price-time limit order book
8 institutional order types
Single-threaded, deterministic
Sub-µs matching · ITCH 5.0 parser

### Execution Realism

Probabilistic maker fills
Queue position · partial fills
Maker/taker fees + rebates
Borrow cost · configurable latency

### Risk Engine

Lock-free kill switch
Token-bucket rate limiter
Position & notional limits
Stale-price guard · VaR histogram

### Strategy Library

Avellaneda-Stoikov market making
OU pairs · TWAP execution
Portfolio mean-reversion
Options MM + Δ-hedge · momentum

### Market Data Feeds

GBM · OU-spread · factor model
Historical CSV tick replay
NASDAQ ITCH 5.0 binary
Live Binance WebSocket (wss)

### Research & Analytics

Signal library · walk-forward
Purged k-fold cross-validation
Sharpe/Sortino/Calmar/MDD · TCA
Tear sheets: JSON · CSV · HTML

**8 order types:** Limit · Market · IOC · FOK · Post-Only · Stop · Stop-Limit · Iceberg

THE DIFFERENTIATOR

# Write Your Own Strategy

```python
# Python — subclass and override callbacks
class MyMM(Strategy):
    def on_book_update(self, book):
        r = mid - q*gamma*sig**2*(T-t)
        self.quote(r - d, r + d)
    def on_fill(self, fill):
        self.inventory += fill.qty

// C++ — compiled in-app to a .dylib
extern "C" Strategy* create() {
    return new MyStrategy();
}
```

### Two ways to author

Python — fast research, full toolkit

C++ — compiled in the Editor tab to a .dylib and hot-loaded, no restart

Both hit the same low-latency core

### Learn by doing

Fork any of the 6 built-in strategies

Embedded terminal (Python preconfigured)

Same binary runs backtest AND live

Your code, real fills, real costs, real risk

LITERATURE & LANDSCAPE

# Related Work & the Gap

### Academic / open-source

QuantLib — pricing only, no engine
Zipline / Backtrader / Backtesting.py
Daily-bar, no tick-level order book
No fees · no queue · no latency model

### Proprietary / commercial

Jane Street / Citadel internal stacks
QuantHouse, Fidessa
Order-book realism — but closed
Expensive, opaque, inaccessible

### The gap QuantSim fills

Open, tick-level C++ limit order book with a Python strategy API, institutional metrics + walk-forward validation, and a live Binance feed with zero-divergence backtest-to-live parity — all in one extensible system.

HOW IT FITS TOGETHER

# System Architecture


*Separation by performance: nanosecond-critical work in C++, productivity work in Python; the GUI reads a mutex-protected snapshot, decoupling render from simulation.*

THE C++ FOUNDATION

# Core C++ Engine

### Limit Order Book

orderbook.cpp

std::map price levels
O(log n) insert / cancel
O(1) best bid / ask
std::mdspan zero-copy views

### Matching Engine

matching.cpp

Price-time priority FIFO
8 institutional order types
Maker / taker fee model
Log-normal latency + queue pos

### Market-Data Feeds

feed.hpp = ITCHParser

NASDAQ ITCH 5.0 binary decode
GBM / OU / factor synthetic
CSV tick replay
Live Binance WebSocket

*Single-threaded, deterministic core — same seed gives bit-identical results.*

REALISTIC EXECUTION

# Matching Engine & Latency Model

### Fill logic

Aggressor walks the passive side
Partial fills allowed
Remainder rests as a limit
Maker rebate · taker fee per fill

### Fee model

maker / taker in basis points
Deducted from cash on fill
Penalises over-trading
Binance-level defaults

### Latency simulation

order, pending queue, then book
Not instant submission
~ log-normal(μ, σ)
clamped to [min_rtt, max_rtt]
Stops trading on unseen prices

RESEARCH INTERFACE

# Python Strategy Layer · pybind11

### The bridge

Exposes LimitOrderBook, MatchingEngine, structs
Strategies authored in Python, run on C++
Marshalling cost paid only at the boundary

### submit_batch() optimisation

Packs N orders into one native call
Amortises Python/C++ context switches
yields sub-microsecond event loops

**Backtest-to-live parity — the core design guarantee**

A strategy subclasses the same interface in both modes.
dlopen() loads the same compiled .dylib at runtime — no code change at the boundary.
Divergence between simulation and production is architecturally impossible.

STRATEGY 1 OF 5

# Avellaneda-Stoikov Market Making

**Reservation price (inventory-skewed mid)**

$$r(s, q, t) = s - q \cdot \gamma \cdot \sigma^2 \cdot (T - t)$$

**Optimal spread**

$$\delta a + \delta b = \gamma\sigma^2(T-t) + (2/\gamma)\cdot\ln(1 + \gamma/\kappa)$$

γ risk aversion · σ volatility · κ book liquidity intensity

Quotes sit around r, not the mid. Long inventory pushes r down, so inventory mean-reverts to zero. Self-regulating.

| 3.12 | −1.8% |
|---|---|
| Sharpe (highest) | Max drawdown |
| **2.45** | **61%** |
| Profit factor | Win rate |

STRATEGIES 2 – 5

# Pairs · TWAP · Portfolio · Options

### 2 · Pairs Trading (OU Stat-Arb)

Spread = $P_A - \beta \cdot P_B$ ; $Z = (\text{spread} - \mu)/\sigma$

Enter $|Z| \geq z$, exit near 0, stop $|Z| \geq z_{stop}$

Sharpe 2.10 · Max DD −1.1% · PF 1.92 · Win 58%

### 3 · TWAP Execution

Splits a large parent order into uniform slices

Not alpha; measured by Implementation Shortfall

Lower realised shortfall than one market order

### 4 · Portfolio Mean-Reversion

Dollar-neutral basket of OU residual edges

Logged run: PnL 1,079.23 · Sharpe 0.04

Max DD −0.98% · PF 1.116 · 731 fills

### 5 · Options MM + Delta Hedge

Quotes around Black-Scholes price; Greeks Δ Γ ν

Hedges underlying to keep net ΣΔ near zero

Sharpe 0.92 · Max DD −1.5% · PF 1.34 · Win 54%

TRUST THE NUMBERS

# Validation Framework

### Walk-Forward

Optimise on a training window
Test out-of-sample on the next
Slide forward, guards overfitting

### Purged k-Fold CV

Purge train data overlapping the test set
Embargo data right after the test set
Kills serial-correlation leakage

### Metric suite

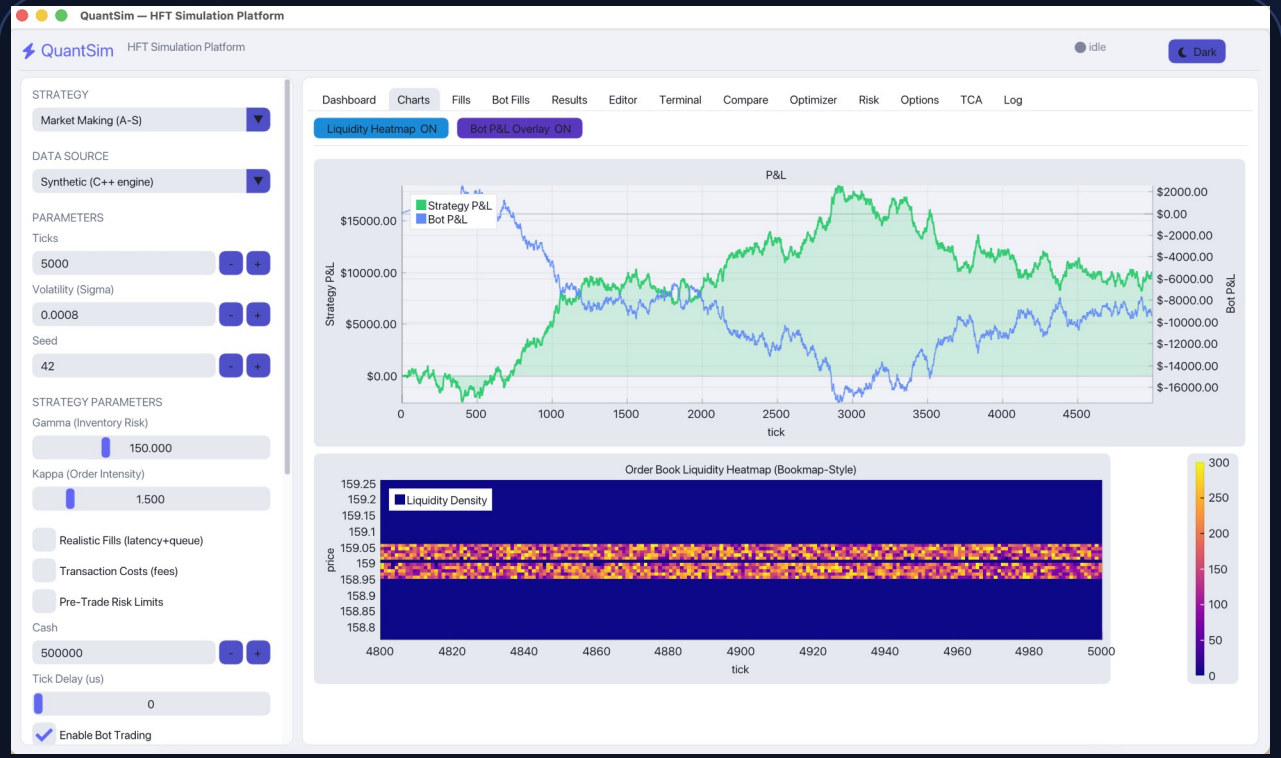
Sharpe = $(\bar{r} - r_f)/\sigma_r \cdot \sqrt{k}$  (k = 1 for tick data — no daily inflation)

Sortino (downside deviation) · Max Drawdown · Calmar (return/|MDD|) · Profit Factor · Win Rate

SEE EVERYTHING

# GUI & Visualization

Dear ImGui + ImPlot + GLFW + OpenGL · immediate-mode, GPU-rendered at 60 fps · 13 analytical tabs.




Charts tab — P&L curve + Bookmap-style liquidity heatmap


Results tab — equity, drawdown & full metric suite

REAL MARKET DATA

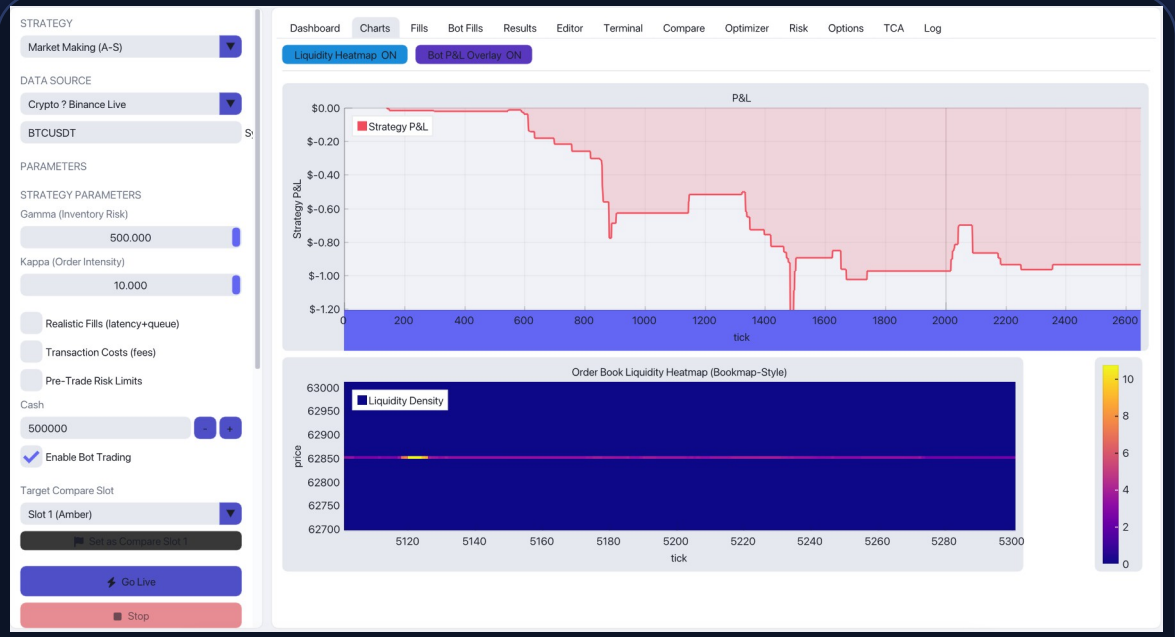
# Live Trading on Binance

## live_trader (C++)

OpenSSL WebSocket (wss://) · real BTCUSDT
book + trade tape · paper trades
Same .dylib via dlopen() for parity

## Session

Avellaneda-Stoikov on BTCUSDT
Around 62,850 USDT
Same metric pipeline as backtest



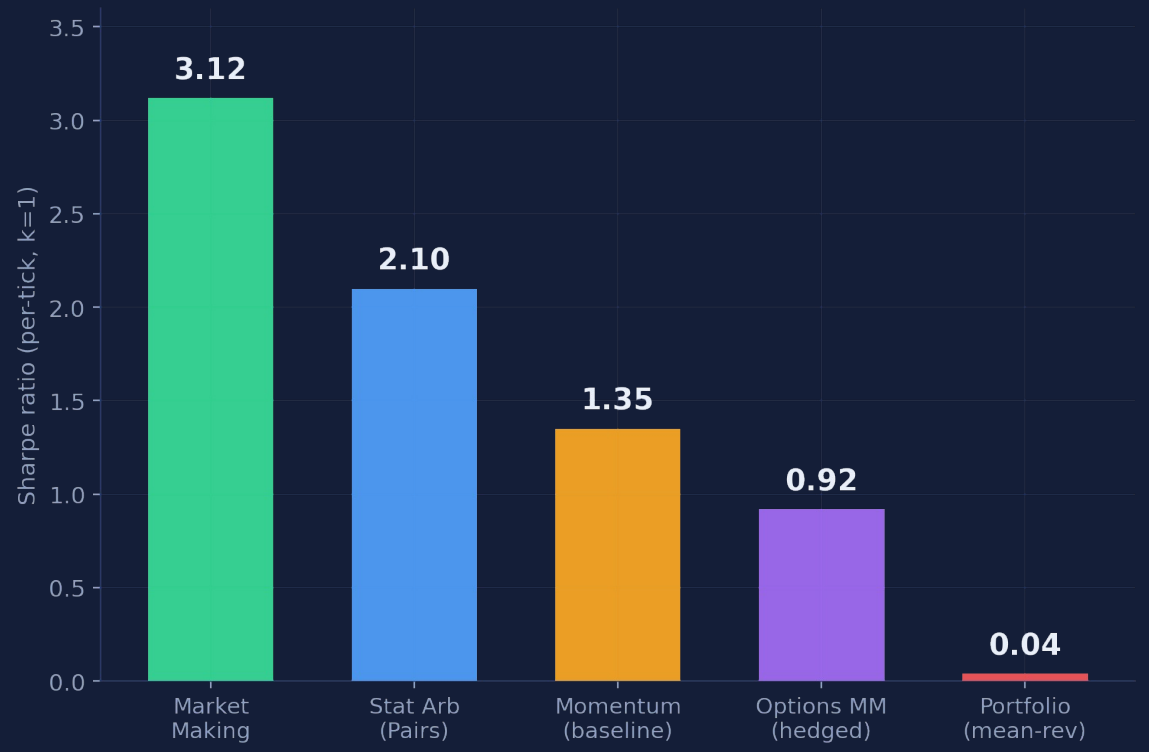

Live BTCUSDT paper-trading session — heatmap + live P&L

*“ The exact same strategy runs in backtest and live with no code change. ”*

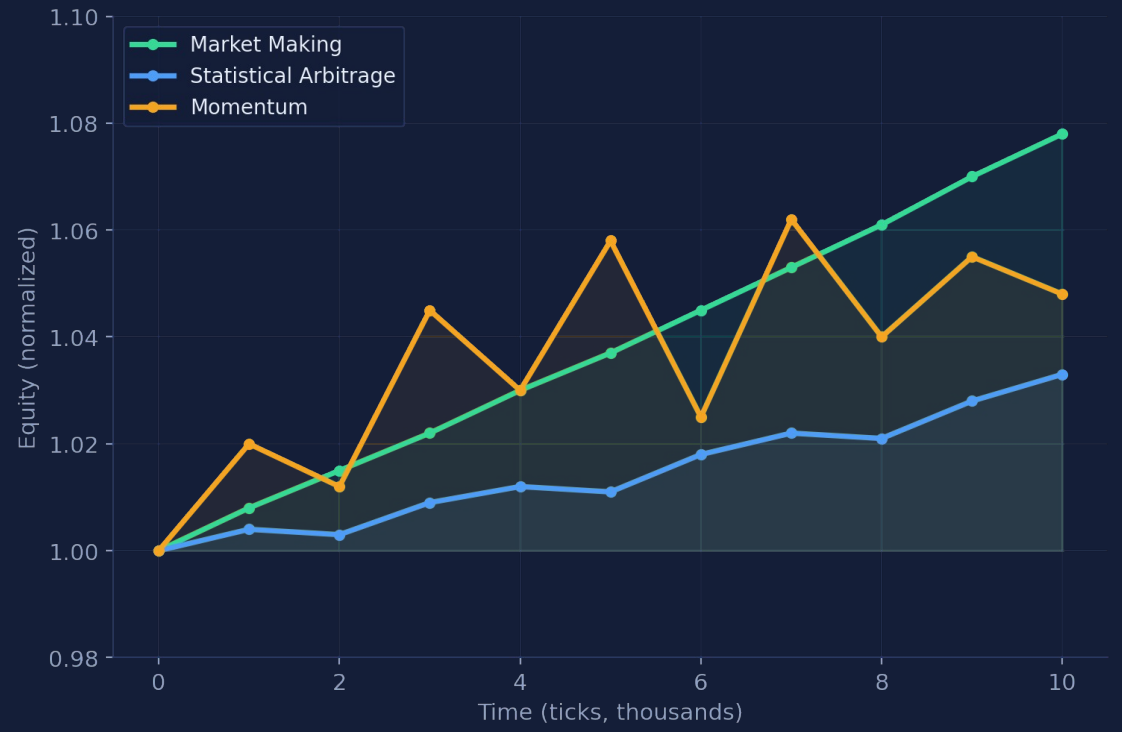
RESULTS

# Strategy Performance

## Sharpe ratio by strategy




## Representative equity curves



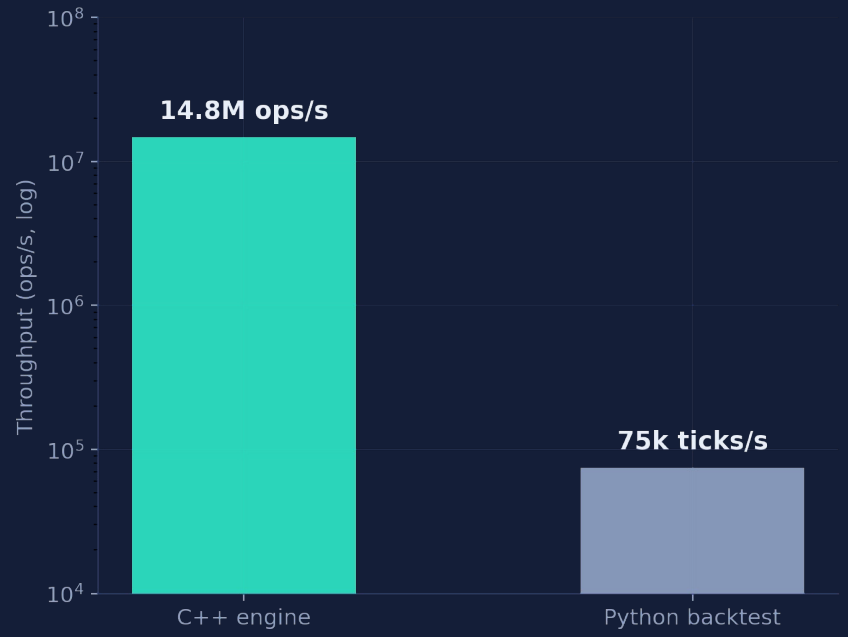

*Synthetic backtest. Market Making compounds steadily; Stat-Arb is smooth but slow; Momentum is volatile. TWAP omitted — execution algo (Implementation Shortfall).*

IT IS FAST

# Engine Benchmarks

## Match latency percentiles


## Throughput (log scale)




**14.8M**
ops / sec

**83 ns**
median latency

**< 5 ns**
kill-switch

**27/27**
tests pass

*Apple M4 Pro · Clang -O3 -march=native. C++ path ~2 orders of magnitude faster than pure Python.*

HONEST BOUNDARIES

# Limitations & Future Work

### Current limitations

Single-venue live feed (Binance)
Short live sessions — small sample
Synthetic data for some portfolio tests
Single-threaded matching core
(deterministic by design)

### Future work

Co-location / cross-venue latency
Multi-exchange live (Coinbase, Kraken)
Options Greeks real-time dashboard
Reinforcement-learning strategy layer
FPGA-accelerated order matching
Longer live runs for robust metrics

# Conclusion

- ✓ C++ matching engine: 14.8M ops/s, 83 ns median, FIFO + fee + latency model
- ✓ 8 order types · 5 strategies + momentum baseline · 27/27 tests pass
- ✓ pybind11 bridge with guaranteed backtest-to-live parity
- ✓ 13-tab GPU GUI with a Bookmap-style liquidity heatmap at 60 fps
- ✓ Live Binance BTCUSDT paper trading via OpenSSL WebSocket

*Fills the gap between closed proprietary stacks and limited academic tools — open, realistic, tick-level, runs on a laptop.*

## Thank You

Abhishek Anand · Shakir Ahmad · Amik Affan | CIT Tatisilwai · Dept. of CSE · 2026